

Índice

RealSteel — IronCode Studio · Manual Técnico

1. Introducción	2
2. Objetivos	3
3. Tecnologías Involucradas	4
3.1. Aprendidas durante el ciclo	4
3.2. Aprendidas durante el desarrollo del proyecto	4
4. Proceso de Desarrollo	6
4.1. Análisis	6
4.2. Desarrollo	6
5. Proceso de Despliegue	10
5.1. Backend — Render	10
5.2. Aplicación Web — Vercel	10
5.3. Landing Page — Vercel	10
5.4. Generación de la APK Android	11
6. Propuesta de Mejora o Trabajos Futuros	12
7. Bibliografía	13

IES ALBARREGAS

Departamento de Informática

Técnico Superior en Desarrollo de Aplicaciones Multiplataforma

REALSTEEL

Manual Técnico

Autores:

Carlos de Tena Muñoz · Fernando Romero Gil · 2ºB DAM

Tutora: Mercedes Martínez Fragoso

Curso Académico: 2025/2026

IRONCODE STUDIO

1. Introducción

En el siguiente documento se detalla la información técnica relativa al proyecto RealSteel desarrollado como Trabajo Final de Grado del ciclo DAM (Desarrollo de Aplicaciones Multiplataforma) en el IES Albarregas, curso 2025/2026.

RealSteel es una aplicación multiplataforma de fitness desarrollada por el equipo IronCode Studio. Permite a los usuarios gestionar rutinas de entrenamiento personalizadas, registrar su progreso con series y pesos, consultar una biblioteca de más de 150 ejercicios, interactuar con un asistente de inteligencia artificial especializado en fitness y localizar gimnasios cercanos.

La aplicación está disponible en dos plataformas:

- **APK Android:** descargable desde la landing page del proyecto, instalable en cualquier dispositivo Android 6.0 o superior.
- **Aplicación web:** desplegada en Vercel (<https://realsteel-henna.vercel.app>), accesible desde cualquier navegador moderno sin instalación.

El sistema se compone de tres componentes diferenciados: el frontend Flutter, el backend Spring Boot y la base de datos PostgreSQL, todos ellos desplegados en producción y operativos.

2. Objetivos

La finalidad principal de RealSteel es proporcionar a los usuarios una herramienta completa, moderna y gratuita para gestionar su entrenamiento personal. Los objetivos técnicos del proyecto son los siguientes:

- **Multiplataforma real:** desarrollar una única base de código Flutter que compile a APK Android y a aplicación web funcional.
- **API REST propia:** implementar un backend con Spring Boot y PostgreSQL, evitando soluciones de terceros como Firebase para mayor control sobre los datos.
- **Despliegue en producción:** tener los tres componentes (frontend, backend, base de datos) operativos y accesibles públicamente.
- **Persistencia de sesión y preferencias:** el usuario no debe hacer login en cada apertura, y sus preferencias de paleta de color deben persistir entre sesiones.
- **Integración de IA:** incorporar un asistente conversacional con Groq (LLaMA 3.3 70B) especializado en fitness y nutrición deportiva.
- **Seguridad básica:** hash BCrypt para contraseñas, validaciones de entrada, credenciales en variables de entorno y recuperación de contraseña por email.
- **Sistema de temas dinámico:** implementar 8 paletas de color (5 oscuras y 3 claras) seleccionables por el usuario y persistentes entre sesiones.

3. Tecnologías Involucradas

3.1. Aprendidas durante el ciclo

Las siguientes tecnologías fueron estudiadas durante el ciclo formativo y aplicadas en el proyecto:

Tecnología	Categoría	Uso en el proyecto
Java 17	Lenguaje	Backend Spring Boot
SQL / PostgreSQL	Base de datos	Persistencia relacional de todos los datos
JSON	Formato de datos	Comunicación entre frontend y backend vía API REST
Git / GitHub	Control de versiones	Repositorio del proyecto y despliegue continuo
HTML5 + CSS3 + JS	Web	Landing page del proyecto
MVC / Arquitectura por capas	Patrón	Organización del backend en controller, service, repository

3.2. Aprendidas durante el desarrollo del proyecto

Las siguientes tecnologías fueron investigadas e implementadas durante el desarrollo de RealSteel, partiendo de conocimiento previo nulo o muy básico:

- **Flutter 3 y Dart:** framework multiplataforma de Google. Aprendido íntegramente durante el proyecto. Permite compilar a Android, iOS y web desde un único código base.
- **Spring Boot 3:** framework Java para APIs REST. Profundización en Spring Security, Spring Data JPA, validaciones y manejo global de excepciones.
- **SharedPreferences:** almacenamiento clave-valor en Flutter para persistir sesión de usuario y paleta de color seleccionada.
- **Render.com:** plataforma cloud para despliegue del backend Spring Boot con integración continua desde GitHub.
- **Vercel:** plataforma para despliegue de la aplicación web Flutter y la landing page estática.
- **Groq API (LLaMA 3.3 70B):** API de IA gratuita utilizada para el chat de asistencia fitness dentro de la aplicación.
- **Brevo API:** servicio de envío de emails HTTP para la recuperación de contraseña. Necesario porque Render bloquea puertos SMTP en el plan gratuito.
- **OpenStreetMap + flutter_map:** librería de mapas de código abierto para la funcionalidad de gimnasios cercanos, sin coste por uso.
- **fl_chart:** librería Flutter para gráficas de progreso del usuario por ejercicio.
- **table_calendar:** componente Flutter para el calendario semanal de planificación de entrenamientos.

4. Proceso de Desarrollo

4.1. Análisis

4.1.1. Diseño visual base

Al inicio del proyecto se realizó el diseño base de la aplicación en Figma (<https://www.figma.com/design/FLI5FymzqVymJIMYGFWIHV/realSteel>). El diseño siguió una estética deportiva oscura con rojo acero como color principal, inspirada en la identidad del proyecto IronCode Studio.

El sistema de diseño implementado incluye 8 paletas de color seleccionables por el usuario, guardadas en SharedPreferences y cargadas antes de que la app arranque:

Paleta	Modo	Acento	Descripción
Steel Red (por defecto)	Oscuro	#C0392B	Rojo acero sobre fondo casi negro
Midnight Purple	Oscuro	#7C3AED	Morado medianoche
Ocean Blue	Oscuro	#2563EB	Azul océano
Forest Green	Oscuro	#16A34A	Verde bosque
Burned Orange	Oscuro	#EA580C	Naranja quemado
Pure White	Claro	#7C3AED	Blanco puro con acento morado
Rose	Claro	#DB2777	Rosa claro
Sky	Claro	#0284C7	Cielo azul claro

4.1.2. Diseño de la base de datos

Para el almacenamiento de los datos se ha optado por PostgreSQL, base de datos relacional gestionada a través de Supabase. Las tablas principales del esquema son:

Tabla	Clave primaria	Relaciones
users	user_id (VARCHAR/UUID)	Referenciada por routines, sessions
exercises	ejercicio_id (BIGINT)	Referenciada por routine_exercises, records
routines	rutina_id (BIGINT)	FK: user_id. Referenciada por routine_exercises, sessions
routine_exercises	id (BIGINT)	FK: rutina_id, ejercicio_id
sessions	session_id (BIGINT)	FK: user_id, rutina_id. Referenciada por records
records	record_id (BIGINT)	FK: session_id, ejercicio_id
supplements	supplement_id (BIGINT)	Tabla independiente

4.1.3. Diagrama de casos de uso

RealSteel presenta un único tipo de usuario (sin roles diferenciados) que puede realizar las siguientes acciones principales:

- Registrarse, iniciar sesión y cerrar sesión.
- Editar su perfil: nombre, email, gimnasio y contraseña.
- Recuperar contraseña por email con código de verificación.
- Planificar su semana asignando grupos musculares a cada día.
- Crear, editar y eliminar rutinas de entrenamiento.
- Asignar ejercicios a rutinas por día de la semana.
- Iniciar un entrenamiento: registrar series, pesos, repeticiones y descansos.
- Consultar la biblioteca de ejercicios (gimnasio y calistenia).
- Ver su progreso: gráficas de evolución por ejercicio.
- Consultar suplementos deportivos con enlace de compra.
- Localizar gimnasios cercanos en el mapa.
- Chatear con el asistente de IA RealSteel AI.
- Cambiar la paleta de color de la aplicación (8 opciones).

4.2. Desarrollo

4.2.1. Diagrama de secuencias

El flujo general de datos en RealSteel sigue una arquitectura de tres capas. El usuario interactúa con la interfaz Flutter, que realiza peticiones HTTP al backend Spring Boot a través de la clase ApiService. El backend procesa la petición, consulta o modifica la base de datos PostgreSQL, y devuelve la respuesta en JSON que Flutter renderiza en pantalla.

Secuencia genérica de una petición:

```

Usuario → Flutter UI → ApiService (HTTP) → Spring Boot
Controller → Service → Repository → PostgreSQL
Usuario ← Flutter UI ← ApiService ← JSON Response
  
```

Para el chat con IA, existe un flujo adicional en paralelo:

```

Flutter (chat_screen.dart) → Groq API (HTTPS directo) → LLaMA 3.3
70B → Respuesta en texto
  
```

4.2.2. Estructura del proyecto

El proyecto se organiza en tres carpetas principales en el repositorio GitHub (<https://github.com/carlosdtina/RealSteel>):

a) Estructura en Flutter (Front/)

El código Flutter sigue una organización por carpetas funcionales dentro de lib/:

Carpeta / Archivo	Descripción
<code>lib/main.dart</code>	Punto de entrada. Define AppColors (getters dinámicos), ThemeController (8 paletas), RS (snackbars globales) y MyApp.
<code>lib/features/</code>	Todas las pantallas de la aplicación (13 archivos .dart).
<code>features/splash_screen.dart</code>	Pantalla de carga: carga sesión y paleta de color antes de navegar.
<code>features/login_screen.dart</code>	Inicio de sesión con animación del Robotín.
<code>features/menu_screen.dart</code>	Pantalla principal: tab Inicio + tab Rutinas. Contiene toda la lógica de entrenamiento en vivo.
<code>features/exercise_screen.dart</code>	Biblioteca de ejercicios con búsqueda y filtros.
<code>features/exercise_detail_screen.dart</code>	Detalle del ejercicio con gráfica de progreso (fl_chart).
<code>features/profile_screen.dart</code>	Edición de perfil y selector de paleta de color.
<code>features/chat_screen.dart</code>	Chat con RealSteel AI (Groq API).
<code>features/gym_map_screen.dart</code>	Mapa de gimnasios (flutter_map + OpenStreetMap).
<code>lib/models/</code>	Modelos de datos: Exercise, Routine, Supplement.
<code>lib/service/api_service.dart</code>	Cliente HTTP centralizado. Todos los métodos de comunicación con el backend.
<code>lib/providers/user_session.dart</code>	Gestión de sesión en memoria y SharedPreferences.
<code>lib/providers/plan_storage.dart</code>	Persistencia del plan semanal por usuario en SharedPreferences.

b) Estructura en el Backend (Back/)

El backend Spring Boot sigue una arquitectura en capas con separación clara entre controladores, servicios, repositorios y entidades:

Paquete	Descripción
<code>controller/</code>	Controladores REST. Reciben peticiones HTTP y delegan en los servicios. Un controlador por entidad.
<code>service/</code>	Lógica de negocio. UserService incluye BCrypt y envío de email. EmailService usa Brevo API HTTP.
<code>repository/</code>	Interfaces JPA. Extienden JpaRepository para operaciones CRUD automáticas sobre PostgreSQL.
<code>entity/</code>	Entidades JPA mapeadas a tablas de la base de datos (7 entidades principales).
<code>dto/</code>	Objetos de transferencia de datos para requests y responses. Desacoplan la API de las entidades internas.

Paquete	Descripción
<code>config/SecurityConfig.java</code>	Spring Security: hash BCrypt, CORS habilitado para todos los orígenes, endpoints públicos.
<code>error/</code>	GlobalExceptionHandler: captura excepciones y devuelve respuestas HTTP correctas (400, 404, 500).

4.2.3. Aspectos destacables de la aplicación

A continuación se detallan las funcionalidades técnicas más relevantes del proyecto:

- **Sistema de temas dinámico:** ThemeController es un ChangeNotifier que gestiona la paleta activa. Los colores se definen como getters estáticos en AppColors (no const), lo que permite que toda la UI se actualice en tiempo real al cambiar de paleta. La paleta se carga con await antes de runApp() para evitar parpadeos.
- **Persistencia de sesión:** UserSession almacena los datos del usuario en memoria (variables estáticas) y en SharedPreferences con claves prefijadas. El splash_screen carga la sesión y la paleta en paralelo con Future.wait antes de navegar a la pantalla correspondiente.
- **Entrenamiento en vivo:** la clase _EntrenamientoScreen dentro de menu_screen.dart gestiona el estado completo del entrenamiento activo. Crea una sesión al iniciarse, permite registrar series con TextEditingController por cada campo, calcula el progreso en tiempo real y guarda todos los records al finalizar mediante una llamada secuencial a la API.
- **Chat con IA sin backend:** chat_screen.dart llama directamente a la API de Groq desde Flutter, sin pasar por el backend propio. El historial de conversación se mantiene en memoria por userId durante la sesión activa.
- **Seguridad de contraseñas:** las contraseñas nunca se almacenan en texto plano. Spring Security con BCryptPasswordEncoder aplica salt automático y múltiples rondas de hash. El endpoint de actualización de perfil solo actualiza la contraseña si el campo viene en el body JSON — si es null, se ignora.
- **Recuperación de contraseña:** UserService genera un código de 6 dígitos almacenado en un ConcurrentHashMap en memoria. Brevo API HTTP envía el email con el código. El código se elimina del mapa tras ser validado correctamente.

4.2.4. Pruebas realizadas

Las pruebas se han realizado de forma práctica y manual durante el desarrollo, sin herramientas externas de testing automatizado. Las principales pruebas realizadas fueron:

- **Autenticación:** registro con datos correctos e incorrectos, login válido e inválido, persistencia de sesión al cerrar y reabrir la app, cierre de sesión y borrado correcto de SharedPreferences.

- **Perfil:** edición de nombre, email y gimnasio sin cambiar contraseña (verificando que el login sigue funcionando), edición con cambio de contraseña, y recuperación de contraseña por email.
- **Rutinas y entrenamiento:** creación y eliminación de rutinas, asignación de ejercicios por día, inicio de entrenamiento, registro de series, temporizador de descanso, finalización y verificación del guardado en base de datos.
- **Temas de color:** cambio de paleta desde el perfil, cierre y reapertura verificando persistencia, comportamiento con ambas paletas claras y oscuras.
- **Estados vacíos:** listas sin rutinas, días sin ejercicios, sin historial — verificación de mensajes de estado vacío en lugar de pantallas en blanco.
- **Web vs Android:** verificación de sesión persistente en Vercel (producción) frente al comportamiento en debug Chrome donde el localStorage no persiste entre reinicios del servidor.

5. Proceso de Despliegue

RealSteel consta de tres componentes desplegados de forma independiente. A continuación se describe el proceso de despliegue de cada uno.

5.1. Backend — Render

El backend Spring Boot está desplegado en Render con integración continua desde GitHub. Cada push a la rama main del repositorio del backend desencadena un redeploy automático.

- Requisitos: cuenta en Render (render.com) y repositorio GitHub con el código Spring Boot.
- En Render: New > Web Service > conectar repositorio GitHub.
- Build Command: `./mvnw clean install -DskipTests`
- Start Command: `java -jar target/realsteel-*.jar`
- Variables de entorno necesarias: DB_URL, DB_USERNAME, DB_PASSWORD, BREVO_API_KEY, SPRING_PROFILES_ACTIVE=production.
- URL resultante: `https://apirealsteel.onrender.com`

Nota: En el plan gratuito, Render pone el servidor en reposo tras 15 minutos de inactividad. El primer acceso puede tardar ~30 segundos en despertar.

5.2. Aplicación Web — Vercel

La aplicación web Flutter se despliega en Vercel. El proceso es el siguiente:

- Ejecutar en local: `flutter build web --release`
- El resultado se genera en la carpeta `Front/build/web/`
- Desde la carpeta Front: `vercel --prod` (requiere Vercel CLI instalado y cuenta vinculada)
- Vercel detecta automáticamente el directorio `build/web` como output.
- URL resultante: `https://realsteel-henna.vercel.app`

5.3. Landing Page — Vercel

La landing page es un proyecto estático HTML+CSS+JS desplegado también en Vercel:

- La carpeta `LandingPage/` contiene `index.html`, `styles.css`, `main.js`, las imágenes del robotín y la APK.
- Desde la carpeta `LandingPage`: `vercel --prod`
- URL resultante: `https://realsteel-landing.vercel.app`

5.4. Generación de la APK Android

Para generar la APK de producción desde Android Studio o línea de comandos:

```
cd Front  
flutter build apk --split-per-abi
```

La APK para dispositivos modernos (arm64) se genera en:

```
Front/build/app/outputs/flutter-apk/app-arm64-v8a-release.apk
```

Esta es la APK recomendada (~19 MB). Compatible con Android 6.0 (API 23) o superior.

6. Propuesta de Mejora o Trabajos Futuros

Las siguientes mejoras están identificadas para versiones futuras de RealSteel:

- **Autenticación con JWT:** implementar tokens de sesión firmados para mayor seguridad y soporte de múltiples dispositivos simultáneos por usuario.
- **Notificaciones push:** recordatorios de entrenamiento programados mediante Firebase Cloud Messaging (FCM).
- **Modo offline:** caché local de rutinas y ejercicios con SQLite para uso de la app sin conexión a internet.
- **Interacción social:** posibilidad de seguir a otros usuarios, compartir rutinas y comparar progresos en un feed.
- **Versión iOS:** compilar y publicar para iOS desde el mismo código Flutter con una cuenta Apple Developer.
- **Calculadora de macros:** integrar control nutricional con objetivos de proteínas, carbohidratos y grasas diarios.
- **Videos y GIFs de ejercicios:** reemplazar imágenes estáticas por demostraciones en vídeo o GIF animado de la técnica correcta.
- **Panel de administración:** interfaz web para gestión de ejercicios, suplementos y estadísticas globales de la aplicación.
- **Tests automatizados:** implementar tests unitarios con JUnit en el backend y widget tests en Flutter.
- **Migración a JWT + refresh tokens:** para que la sesión web persista de forma segura sin depender de localStorage.

7. Bibliografía

Documentación oficial de Flutter y Dart:

- <https://flutter.dev/docs>
- <https://dart.dev/guides>

Documentación oficial de Spring Boot y Spring Security:

- <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- <https://docs.spring.io/spring-security/reference/>

Documentación de PostgreSQL:

- <https://www.postgresql.org/docs/>

Groq API (LLaMA 3.3):

- <https://console.groq.com/docs/openai>

Brevo API (envío de emails):

- <https://developers.brevo.com/>

Render (despliegue backend):

- <https://render.com/docs>

Vercel (despliegue web):

- <https://vercel.com/docs>

OpenStreetMap y flutter_map:

- <https://docs.fleatlet.dev/>

fl_chart (gráficas):

- https://pub.dev/packages/fl_chart

Resolución de problemas y consultas técnicas:

- <https://stackoverflow.com/>
- <https://pub.dev/> (paquetes Flutter)
- <https://github.com/carlosdtna/RealSteel> (repositorio del proyecto)